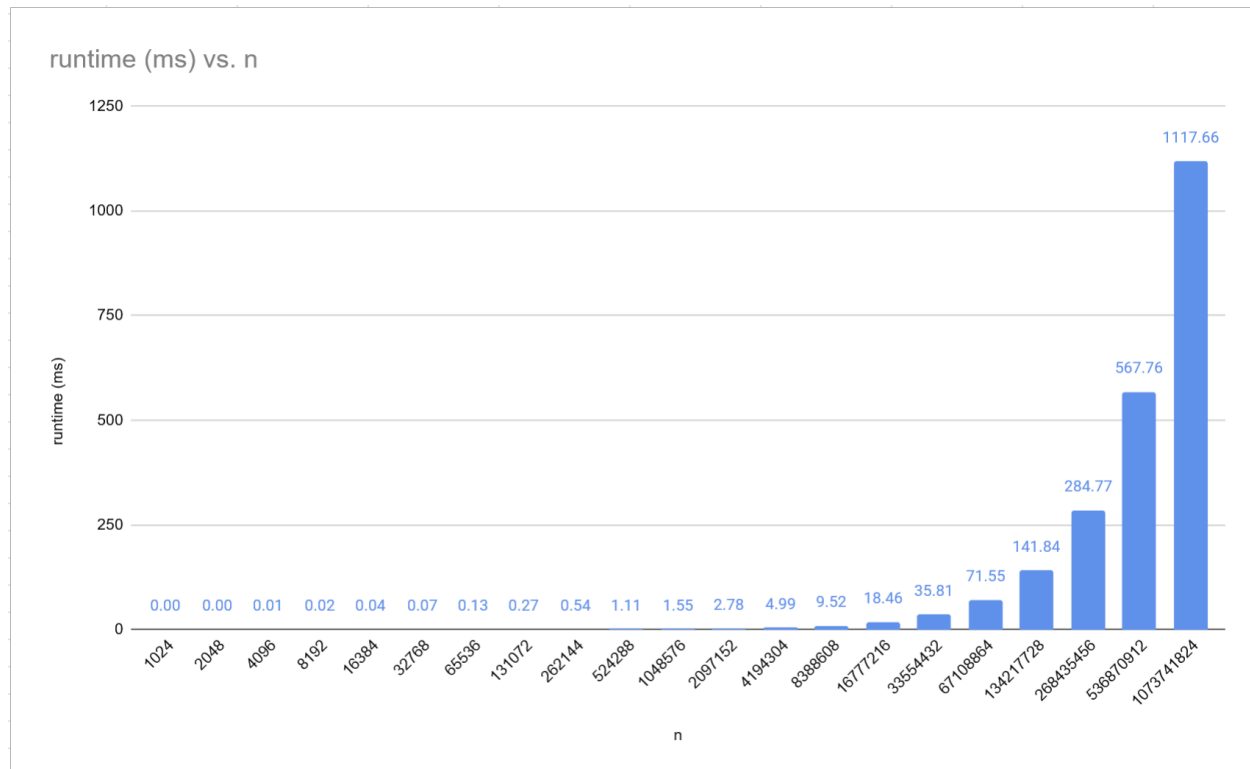


Fall 2026 | PA 1

Frey (Henry) Wysong-Grass

GitHub repo: <https://github.com/Henrywwg/repo759>HW01 repo: <https://github.com/Henrywwg/repo759/tree/main/HW02>

1) Here is the plot acquired during the runs.



2) Done. See GitHub repo.

3) mmul1, 2 and 3 all have differing performance due to how we index into their arrays. mmul1 operates well as we iterate over rows in A, columns in B and then iterate through both the specified column and row, summing the value and writing to C. mmul2 has slightly better performance. Looking at how we index into the array, we see we actually are doing a stride of only a single value instead of whole rows (like in mmul1). This means we are always doing sequential accesses within the innermost loop. In mmul3, we perform the worst since in the innermost loop we take huge strides each loop and jump entire rows/columns in 2 of the 3 arrays.

mmul1 and mmul4 both perform the same basic operation, but mmul4 is slightly slower. As we discussed in class, this is likely due to how a vector is allocated. Although we index it the same as a 1D array, it is not guaranteed, nor likely, to be allocated as a contiguous block of memory like one would expect when requesting memory. This means that indexing even sequential elements in the vector could lead to non-sequential memory accesses, slightly slowing down our program.

